Web Technologies — Unit I & Unit II Comprehensive Study Notes

> ***Subject Type:*** *Programming*
> ***Units Covered:*** *Unit I (HTML, CSS, XML, XHTML) & Unit II (JavaScript, JSP)*
> ***Exam Ready:*** *Yes*

Detailed Syllabus

UNIT-I (15 Marks)

HTML: Basic Syntax, Standard HTML Document Structure, Basic Text Markup, HTML styles, Elements, Attributes, Heading, Layouts, I frames, Images, Hypertext Links, Lists, Tables, Forms, Dynamic HTML

CSS: Need for CSS, introduction to CSS, basic syntax and structure, using CSS, background images, colors, and properties, manipulating texts, using fonts, borders, boxes, margins, padding lists, positioning using CSS, CSS2, The Box Model

XML: Document Type Definition (DTD), XML schemas, Document object model, Parsers -DOM, and SAX

XHTML: Introduction to XHTML, XML, Meta tags, Character entities, frames, and frame sets

---

UNIT-II (15 Marks)

JavaScript: Client-side scripting, Introduction to JavaScript, Objects, Primitives, Operations and Expressions, Control Statements, Arrays, Functions, Constructors, JavaScript Objects, JavaScript own objects, the DOM and web browser environments, forms and validations

JSP: The Anatomy of a JSP Page, JSP Processing, Declarations, Directives, Expressions, Code Snippets, implicit objects, Using Beans in JSP Pages, Using Cookies and session for session tracking, connecting to database in JSP

Table of Contents

1. Unit I: HTML, CSS, XML, XHTML
2. Unit II: JavaScript and JSP

Unit I: HTML, CSS, XML, XHTML

1. HTML Fundamentals

1.1 Definition and Importance

What is HTML?

HTML (Hypertext Markup Language) is the standard markup language used to create and design web pages. It forms the foundation of every website on the internet. Think of HTML as the skeleton or structure of a web page - it provides the basic framework upon which all other web technologies are built.

Why HTML is Important:

1. **Universal Standard:** HTML is the universal language of the web. Every browser understands HTML, making it the most important technology for web development.
2. **Search Engine Optimization:** Proper use of HTML elements helps search engines understand and index web content effectively.
3. **Accessibility:** Semantic HTML provides better accessibility for users with disabilities using screen readers.
4. **Foundation for Other Technologies:** CSS and JavaScript build upon HTML to add styling and interactivity.

The Evolution of HTML:

- HTML 1.0 (1991): First version with basic tags
- HTML 2.0 (1995): Added forms and tables
- HTML 3.2 (1997): Added tables, applets, text flow around images
- HTML 4.01 (1999): CSS support, accessibility improvements
- HTML5 (2014): Semantic elements, multimedia support, canvas, local storage

1.2 Basic Syntax

- **Tags:** Tags are enclosed in angle brackets (`<` and `>`). Tags usually come in pairs: opening tag `<tagname>` and closing tag `</tagname>`
- **Attributes:** Provide additional information, specified in the start tag as name-value pairs
- **Whitespace:** Extra spaces, tabs, and new lines within HTML code are generally ignored
- **Comments:** Begin with `<!--` and end with `-->`
- **Case Sensitivity:** HTML is not case sensitive; however, it is a good practice to use lowercase

Key Concepts in HTML Syntax:

1. **Tag Structure:** HTML tags define elements. Most tags come in pairs - an opening tag and a closing tag. The content goes between these tags.
2. **Nesting:** Tags can be nested inside other tags. However, they must be properly nested (inner tags must close before outer tags).
3. **Self-Closing Tags:** Some tags don't require closing tags. These are called void elements: `<br>` , `<hr>` , `<img>` , `<input>` , `<meta>` , `<link>` .
4. **Attributes:** Attributes provide additional information about elements. They always go in the opening tag and consist of name="value" pairs.

1.3 Standard HTML Document Structure

Every HTML document must follow a standard structure to ensure proper rendering across different browsers. This structure is mandatory for valid HTML documents.

Explanation of Each Component:

1. **`<!DOCTYPE html>`** : This is the document type declaration (DTD). It tells the browser which version of HTML the document uses. For HTML5, we simply write `<!DOCTYPE html>` . This declaration must be the very first thing in your HTML document.
2. **`<html>`** : This is the root element that wraps all content in the document. Everything else goes inside this element. The `lang` attribute can specify the language: `<html lang="en">` .
3. **`<head>`** : This element contains metadata about the document. Information in the head is not displayed on the page but includes:

- Character encoding
- Page title
- CSS styles
- JavaScript files
- Meta tags for SEO
- Viewport settings for responsive design

4. `<title>` : Sets the text that appears in the browser's title bar or tab. This is important for user experience and search engine optimization.
5. `<body>` : Contains all visible content - text, images, links, tables, forms, etc.

```html
<!DOCTYPE html>
<html>
<head>
    <title>Page Title</title>
    <meta charset="UTF-8">
    <meta name="description" content="Description">
    <meta name="keywords" content="keyword1, keyword2">
    <meta name="author" content="Author Name">
    <link rel="stylesheet" href="style.css">
    <script src="script.js"></script>
</head>
<body>
    <h1>Main Heading</h1>
    <p>This is a paragraph.</p>
</body>
</html>
```

Document Structure Elements:

Element	Description
`<!DOCTYPE html>`	Declares document type and version (HTML5)
`<html>`	Root element of the document
`<head>`	Contains meta-information (title, charset, styles, scripts)
`<title>`	Specifies the document title shown in browser tab
`<body>`	Contains visible content (text, images, links, etc.)
`<meta>`	Provides metadata about the HTML document
`<link>`	Defines relationship between document and external resource
`<script>`	Embeds or references JavaScript
`<style>`	Contains CSS styling

1.4 HTML Elements

Understanding Block-Level vs Inline Elements:

Block-Level Elements:

- Start on a new line and take up the full width available
- They create "blocks" of content
- Can contain other block elements and inline elements
- Examples: `<div>`, `<p>`, `<h1>` to `<h6>`, `<ul>`, `<ol>`, `<li>`, `<table>`, `<form>`, `<header>`, `<footer>`, `<section>`, `<article>`, `<nav>`, `<main>`

Why Block Elements Matter:

1. They create visual structure on the page
2. They define the layout and organization of content
3. Search engines use them to understand document structure
4. They help with accessibility

Inline Elements:

- Do not start on a new line
- Only take up as much width as necessary
- Cannot contain block elements
- Typically used for text formatting or small UI elements
- Examples: `<span>`, `<a>`, `<strong>`, `<em>`, `<img>`, `<br>`, `<input>`, `<label>`, `<select>`, `<textarea>`, `<button>`

Important Note: HTML5 allows both block and inline elements to be used more flexibly, but understanding the distinction helps with CSS styling and layout.

1.5 HTML Attributes

Deep Dive into Attributes:

Attributes provide additional information about HTML elements. They are always specified in the opening tag and consist of name-value pairs.

Essential Attributes Explained:

1. `id` **Attribute:**
   - Provides a unique identifier for an element
   - Each id must be unique within the document
   - Used by CSS for styling and JavaScript for manipulation
   - Example: `<div id="header">`
2. `class` **Attribute:**
   - Assigns one or more classes to an element
   - Unlike id, class can be reused
   - Multiple classes can be assigned with spaces: `class="highlight important"`
   - Example: `<p class="intro text-large">`
3. `src` **Attribute:**
   - Specifies the source URL for images, videos, audio
   - For images: `<img src="photo.jpg">`

- For scripts: `<script src="app.js">`
- Can use absolute or relative URLs

4. `href` **Attribute:**
   - Specifies the URL for anchor links
   - Essential for navigation
   - Example: `<a href="page.html">`
   - Can link to URLs, files, or page sections

5. `alt` **Attribute:**
   - Provides alternative text for images
   - Critical for accessibility (screen readers)
   - Displayed when image fails to load
   - Important for SEO

6. `title` **Attribute:**
   - Provides additional information about an element
   - Shows as tooltip on hover
   - Example: `<span title="More info">text</span>`

7. `disabled` **Attribute:**
   - Disables an input element
   - Prevents user interaction
   - Visual indication that element is inactive

8. `required` **Attribute:**
   - Makes form field mandatory
   - Browser performs validation
   - Example: `<input required>`

9. `placeholder` **Attribute:**
   - Shows example text in empty input
   - Disappears when user types
   - Provides user guidance

1.6 Basic Text Markup

Headings: Six levels (`<h1>` to `<h6>`), `<h1>` is most important

```
<h1>Main Heading</h1>
<h2>Subheading</h2>
<h3>Sub-subheading</h3>
```

Paragraphs and Text Formatting:

```
<p>This is a paragraph.</p>
<em>Emphasized text (italic)</em>
<strong>Strong text (bold)</strong>
<i>Italic text</i>
<b>Bold text</b>
<u>Underlined text</u>
<br> <!-- Line break -->
<hr> <!-- Horizontal rule -->
<mark>Highlighted text</mark>
<del>Deleted text</del>
<ins>Inserted text</ins>
<sup>Superscript</sup>
<sub>Subscript</sub>
```

1.7 Lists

Ordered List (Numbered):

```
<ol type="1" start="1">
    <li>First item</li>
    <li>Second item</li>
</ol>
```

- `type` values: "1", "A", "a", "I", "i"

Unordered List (Bulleted):

```
<ul type="disc">
    <li>First item</li>
    <li>Second item</li>
</ul>
```

- `type` values: "disc", "circle", "square"

Definition List:

```
<dl>
    <dt>Term</dt>
    <dd>Definition</dd>
</dl>
```

1.8 Tables

```
<table border="1" cellpadding="5" cellspacing="5">
    <caption>Table Title</caption>
    <thead>
        <tr>
            <th>Header 1</th>
            <th>Header 2</th>
            <th>Header 3</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>Row 1, Col 1</td>
            <td>Row 1, Col 2</td>
            <td rowspan="2">Row 1 & 2, Col 3</td>
        </tr>
        <tr>
            <td colspan="2">Row 2, Col 1 & 2</td>
        </tr>
    </tbody>
    <tfoot>
        <tr>
            <td>Footer 1</td>
            <td>Footer 2</td>
            <td>Footer 3</td>
        </tr>
    </tfoot>
</table>
```

Table Attributes: `border`, `cellpadding`, `cellspacing`, `colspan`, `rowspan`, `width`, `height`

1.9 Forms

```html
<form action="/submit" method="post" enctype="multipart/form-data">
    <fieldset>
        <legend>Personal Information</legend>

        <label for="username">Username:</label>
        <input type="text" id="username" name="username" required>

        <label for="email">Email:</label>
        <input type="email" id="email" name="email" required>

        <label for="password">Password:</label>
        <input type="password" id="password" name="password" minlength="8" required>

        <label for="gender">Gender:</label>
        <input type="radio" name="gender" value="male"> Male
        <input type="radio" name="gender" value="female"> Female

        <label for="country">Country:</label>
        <select id="country" name="country">
            <option value="">Select Country</option>
            <option value="usa">USA</option>
            <option value="uk">UK</option>
            <option value="india">India</option>
        </select>

        <label for="bio">Bio:</label>
        <textarea id="bio" name="bio" rows="4" cols="50"></textarea>

        <label>
            <input type="checkbox" name="terms" required> I agree to terms
        </label>

        <input type="submit" value="Submit">
        <input type="reset" value="Reset">
        <input type="button" value="Custom Button">
    </fieldset>
</form>
```

Input Types: `text`, `password`, `email`, `number`, `tel`, `url`, `date`, `time`, `datetime-local`, `color`, `range`, `file`, `hidden`, `checkbox`, `radio`, `submit`, `reset`, `button`

1.10 Images and Links

Images:

```html
<img src="image.jpg" alt="Description" width="300" height="200" loading="lazy">
<figure>
    <img src="image.jpg" alt="Description">
    <figcaption>Caption for the image</figcaption>
</figure>
```

Hypertext Links:

```html
<a href="https://www.example.com" target="_blank" rel="noopener noreferrer">Visit
Example</a>
<a href="#section-id">Jump to Section</a>
<a href="mailto:email@example.com">Send Email</a>
<a href="tel:+1234567890">Call Us</a>
```

1.11 Frames (Deprecated)

```html
<!-- Deprecated frameset (not recommended) -->
<frameset cols="25%,75%">
    <frame src="menu.html" name="menu">
    <frame src="content.html" name="content">
    <noframes>
        <body>Your browser does not support frames.</body>
    </noframes>
</frameset>

<!-- Modern alternative -->
<iframe src="content.html" width="600" height="400" frameborder="0"></iframe>
```

1.12 Semantic HTML Elements

```html
<header>
    <nav>
        <a href="/">Home</a>
        <a href="/about">About</a>
        <a href="/contact">Contact</a>
    </nav>
</header>

<main>
    <article>
        <section>
            <h1>Article Title</h1>
            <p>Article content...</p>
        </section>
    </article>

    <aside>
        <h2>Related Content</h2>
        <p>Sidebar content...</p>
    </aside>
</main>

<footer>
    <p>&copy; 2024 Company Name</p>
</footer>
```

---

2. CSS (Cascading Style Sheets)

2.1 Need for CSS - Why CSS is Essential

Understanding CSS:

CSS (Cascading Style Sheets) is a stylesheet language used to describe the presentation of a document written in HTML. CSS separates content from presentation, allowing developers to control layout, colors, fonts, and visual effects.

Why CSS is Important:

1. **Separation of Concerns:**
    - CSS separates content (HTML) from presentation (styling)
    - Makes code cleaner and more maintainable
    - Allows parallel development (designer and developer can work independently)

2. **Consistency Across Pages:**

    - Apply uniform styling across multiple pages
    - Change style in one place updates entire website
    - Reduces code duplication

3. **Responsive Design:**
    - Create layouts that adapt to different screen sizes
    - Essential for mobile-friendly websites
    - Media queries enable device-specific styling

4. **Faster Page Loading:**
    - CSS files are cached by browsers
    - Reduces HTML file size
    - Improves website performance

5. **Enhanced User Experience:**
    - Animations and transitions engage users
    - Better visual hierarchy improves readability
    - Professional appearance builds trust

6. **Accessibility:**
    - Improve readability for users with disabilities
    - Support for screen readers
    - Better contrast and text sizing options

7. **Search Engine Optimization:**
    - Clean HTML improves search engine crawling
    - Faster loading times boost rankings
    - Semantic HTML with CSS styling helps SEO

The Evolution of CSS:

- CSS1 (1996): Basic styling capabilities
- CSS2 (1998): Added positioning, z-index, media types
- CSS3 (1999-present): Modular approach, animations, flexbox, grid, transitions

2.2 CSS Syntax

```
/* Selector */
selector {
    property1: value1;
    property2: value2;
    /* Comments */
}
```

Types of Selectors:

```
/* Element Selector - selects all elements of a given type */
p { color: blue; }

/* Class Selector - selects elements with a specific class */
.highlight { background-color: yellow; }

/* ID Selector - selects a unique element with the specified ID */
#header { font-size: 24px; }

/* Attribute Selector - selects elements with a specific attribute */
input[type="text"] { border: 1px solid black; }

/* Descendant Selector - selects elements nested inside another element */
ul li { list-style-type: square; }

/* Child Selector - selects direct children of an element */
ul > li { margin: 5px; }

/* Pseudo-class - selects elements based on their state */
a:hover { color: red; }

/* Pseudo-element - styles specific parts of an element */
p::first-line { font-weight: bold; }

/* Grouping - applies same styles to multiple selectors */
h1, h2, h3 { color: red; }
```

2.3 CSS Properties

Background:

```
.container {
    background-color: #f0f0f0;
    background-image: url("bg.jpg");
    background-repeat: repeat;
    background-position: center top;
    background-size: cover;
    background-attachment: fixed;
    /* Shorthand - combines all background properties into one */
    background: #f0f0f0 url("bg.jpg") no-repeat center top;
}
```

Text and Font:

```
body {
    font-family: Arial, sans-serif;
    font-size: 16px;
    font-weight: normal;
    font-style: normal;
    color: #333;
    text-align: left;
    text-decoration: none;
    text-transform: capitalize;
    text-indent: 30px;
    line-height: 1.5;
    letter-spacing: 2px;
    word-spacing: 5px;
}
```

Borders:

```
.box {
    border: 2px solid #000;
    border-width: 2px;
    border-style: solid;
    border-color: #000;
    border-radius: 10px;
    box-shadow: 3px 3px 5px rgba(0, 0, 0, 0.5);
}
```

Margins and Padding:

```
.element {
    margin: 10px;
    margin-top: 10px;
    margin-right: 20px;
    margin-bottom: 10px;
    margin-left: 20px;
    padding: 20px;
    padding-top: 10px;
}
```

2.4 The Box Model

```
┌──────────────────────────────────────┐
│           Margin                     │
│  ┌──────────────────────────────────┐ │
│  │         Border                   │ │
│  │  ┌────────────────────────────┐  │ │
│  │  │       Padding              │  │ │
│  │  │  ┌──────────────────────┐  │  │ │
│  │  │  │     Content          │  │  │ │
│  │  │  │                      │  │  │ │
│  │  │  └──────────────────────┘  │  │ │
│  │  └────────────────────────────┘  │ │
│  └──────────────────────────────────┘ │
└──────────────────────────────────────┘
```

```
.box {
    width: 200px;
    height: 100px;
    padding: 20px;
    border: 2px solid black;
    margin: 10px;
    box-sizing: content-box; /* Default - width includes only content */
    /* or */
    box-sizing: border-box; /* Includes padding and border in width */
}
```

2.5 Positioning in CSS

```
.static { position: static; }

.relative {
    position: relative;
    top: 20px;
    left: 10px;
}

.absolute {
    position: absolute;
    top: 50px;
    left: 50px;
}

.fixed {
    position: fixed;
    top: 0;
    right: 0;
}

.sticky {
    position: sticky;
    top: 10px;
}

/* Z-index */
.high { z-index: 10; position: relative; }
.low { z-index: 5; position: relative; }

/* Float */
.left { float: left; }
.right { float: right; }
.clear { clear: both; }
```

2.6 CSS Layouts

CSS Grid:

```
.container {
    display: grid;
    grid-template-columns: 1fr 1fr 1fr;
    grid-template-rows: auto;
    grid-gap: 20px;
    grid-template-areas:
        "header header header"
        "sidebar content content"
        "footer footer footer";
}

.item {
    grid-column: 1 / 3; /* Span 2 columns - from line 1 to 3 */
    grid-row: 1 / 2;
}
```

Flexbox:

```
.container {
    display: flex;
    flex-direction: row;
    justify-content: space-between;
    align-items: center;
    flex-wrap: wrap;
}

.item {
    flex-grow: 1;
    flex-shrink: 0;
    flex-basis: 200px;
    /* Shorthand: flex-grow flex-shrink flex-basis */
}
```

2.7 CSS3 Features

Gradients:

```
.gradient-linear {
    background-image: linear-gradient(to right, #ff0000, #0000ff);
}

.gradient-radial {
    background-image: radial-gradient(circle, #ff0000, #0000ff);
}

.gradient-angle {
    background-image: linear-gradient(45deg, #ff0000, #0000ff);
}
```

Transitions:

```
.button {
    transition: all 0.3s ease;
}
.button:hover {
    background-color: #333;
    transform: scale(1.1);
}
```

Animations:

```
@keyframes slideIn {
    from { transform: translateX(-100%); }
    to { transform: translateX(0); }
}

.animation {
    animation: slideIn 1s ease-in-out;
}
```

Media Queries (Responsive Design):

```
/* Mobile First - base styles for small screens */
.container {
    width: 100%;
}

/* Tablet - styles for screens 768px and wider */
@media (min-width: 768px) {
    .container {
        width: 750px;
    }
}

/* Desktop - styles for screens 1024px and wider */
@media (min-width: 1024px) {
    .container {
        width: 960px;
    }
}

/* Print - styles for printing */
@media print {
    body {
        font-size: 12pt;
    }
}
```

---

3. XML (eXtensible Markup Language)

3.1 Definition

XML (eXtensible Markup Language) is a markup language that defines rules for encoding documents in a format that is both human-readable and machine-readable. It is used for data exchange between different systems.

3.2 XML Document Structure

```xml
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="style.xsl"?>
<!-- Comment -->
<books>
    <book id="1">
        <title>The Great Gatsby</title>
        <author>
            <firstname>F. Scott</firstname>
            <lastname>Fitzgerald</lastname>
        </author>
        <year>1925</year>
        <price currency="USD">10.99</price>
    </book>
    <book id="2">
        <title>To Kill a Mockingbird</title>
        <author>Harper Lee</author>
        <year>1960</year>
    </book>
</books>
```

3.3 XML Rules (Well-Formedness)

- Must have a single root element
- All elements must be properly nested
- Attribute values must be quoted
- Tags are case-sensitive
- All tags must have closing tags (or be self-closing)

---

4. DTD (Document Type Definition)

4.1 Definition

DTD is a formal specification that defines the structure, elements, attributes, and relationships within an XML document.

4.2 DTD Syntax

```
<!DOCTYPE employees [
    <!ELEMENT employees (employee*)>
    <!ELEMENT employee (name, department, salary)>
    <!ATTLIST employee
        id ID #REQUIRED
        status (active|inactive) "active">
    <!ELEMENT name (#PCDATA)>
    <!ELEMENT department (#PCDATA)>
    <!ELEMENT salary (#PCDATA)>
    <!ENTITY copyright "Copyright 2024">
]>
```

Key DTD Declarations:

- `<!ELEMENT>` : Defines elements and content model
- `<!ATTLIST>` : Defines attributes
- `<!ENTITY>` : Defines entities
- `#PCDATA` : Parsed Character Data
- `#REQUIRED` : Attribute must be present
- `#IMPLIED` : Attribute is optional
- `#FIXED` : Attribute has fixed value

---

5. XML Schema (XSD)

5.1 Definition

XML Schema Definition (XSD) is a more powerful alternative to DTD for defining structure and constraints of XML documents.

5.2 XML Schema Example

```xml
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

    <xs:element name="employees">
        <xs:complexType>
            <xs:sequence>
                <xs:element ref="employee" minOccurs="0" maxOccurs="unbounded"/>
            </xs:sequence>
        </xs:complexType>
    </xs:element>

    <xs:element name="employee">
        <xs:complexType>
            <xs:sequence>
                <xs:element name="name" type="xs:string"/>
                <xs:element name="department" type="xs:string"/>
                <xs:element name="salary" type="xs:decimal"/>
            </xs:sequence>
            <xs:attribute name="id" type="xs:ID" use="required"/>
            <xs:attribute name="status" type="xs:string" default="active"/>
        </xs:complexType>
    </xs:element>

</xs:schema>
```

5.3 DTD vs XML Schema Comparison

Feature	DTD	XML Schema
Syntax	Non-XML	XML
Data Types	Limited	Rich data types
Namespace Support	No	Yes
Validation	Less expressive	More powerful
Inheritance	No	Yes
Modularity	Limited	High
Extensibility	Less extensible	More extensible

6. DOM and SAX Parsers

6.1 DOM (Document Object Model)

- **Type:** Tree-based parser
- **Memory:** Loads entire XML into memory
- **Access:** Random access to elements
- **Best For:** Small to medium XML files
- **Advantages:** Easy to navigate, random access
- **Disadvantages:** Memory intensive for large files

6.2 SAX (Simple API for XML)

- **Type:** Event-based parser
- **Memory:** Sequential reading, doesn't load into memory
- **Access:** Forward-only access
- **Best For:** Large XML files
- **Advantages:** Low memory usage, fast
- **Disadvantages:** Complex to implement, no random access

---

7. XHTML

7.1 Definition

XHTML (Extensible HyperText Markup Language) is a stricter and more xml-compliant version of HTML.

7.2 Differences from HTML

1. All tags must be lowercase
2. All tags must be properly closed
3. Attribute values must be quoted
4. Nesting must be correct
5. Documents must have a DOCTYPE declaration

7.3 XHTML Template

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8"/>
    <title>XHTML Document</title>
</head>
<body>
    <p>Content here</p>
</body>
</html>
```

---

8. Dynamic HTML (DHTML)

8.1 Definition

DHTML is the combination of HTML, CSS, and JavaScript to create interactive and dynamic web content.

8.2 Event Handling

```
<button onclick="alert('Clicked!')">Click Me</button>
<div onmouseover="this.style.background='yellow'"
onmouseout="this.style.background='white'">Hover Me</div>
<input onchange="console.log(this.value)" onfocus="this.style.border='2px solid blue'">
<form onsubmit="return validateForm()">
```

8.3 DOM Manipulation

```
// Create element - create new DOM nodes
var p = document.createElement("p");
p.textContent = "New paragraph";
document.body.appendChild(p);

// Modify element - change content, styles, or attributes
document.getElementById("myId").innerHTML = "<strong>New content</strong>";
element.style.color = "blue";
element.classList.add("active");

// Remove element - delete DOM nodes
element.remove();

// Event listener - handle user interactions
element.addEventListener("click", function(event) {
    console.log(event.target);
});
```

Unit I: Practice Questions

1. **[Easy]** Explain the difference between block-level and inline elements in HTML with examples.
2. **[Medium]** Create an HTML form with validation for username (min 5 chars), email, password (min 8 chars), and confirm password.
3. **[Hard]** Design a responsive webpage layout using CSS Grid and Flexbox that works on mobile, tablet, and desktop.

Unit I: Quick Reference Card

Concept	Syntax/Formula	Example
HTML Tag	`<tagname>`	`<p>`, `<div>`
CSS Selector	`element { }`	`p { color: blue; }`
Box Model	margin → border → padding → content	width includes all
XML Declaration	`<?xml version="1.0"?>`	First line of XML
CSS Positioning	`position: absolute`	Relative to ancestor
CSS Grid	`display: grid`	2D layout
Flexbox	`display: flex`	1D layout
DOM	Tree structure	getElementById
DTD	Document Type Definition	XML validation
XHTML	Strict HTML	XML syntax

Unit II: JavaScript and JSP

1. JavaScript Fundamentals

1.1 Definition

JavaScript is a client-side scripting language that enables interactive web pages. It runs in the browser and can manipulate HTML content, handle events, and validate forms.

1.2 Variables (var, let, const)

```
var name = "John";          // Function-scoped, can be redeclared
let age = 25;              // Block-scoped, can be reassigned
const PI = 3.14159;        // Block-scoped, cannot be reassigned

// Differences
var x = 10;
if (true) {
    var x = 20;  // Same variable
}
console.log(x);  // 20

let y = 10;
if (true) {
    let y = 20;  // Different variable
}
console.log(y);  // 10
```

1.3 Data Types

Primitives:

```
var num = 42;              // Number (includes integers and floats)
var str = "Hello";         // String
var bool = true;           // Boolean
var empty = null;          // Null
var notDefined;            // Undefined
var symbol = Symbol('id'); // Symbol
var bigInt = 9007199254740991n; // BigInt
```

Objects:

```
var obj = { name: "John", age: 25 };
var arr = [1, 2, 3, 4, 5];
var func = function() { return "Hello"; };
```

1.4 Operations and Expressions

```
// Arithmetic operators: +, -, *, /, %, **
var sum = 10 + 5;          // 15
var product = 10 * 5;      // 50
var remainder = 10 % 3;    // 1
var power = 2 ** 3;        // 8

// Comparison operators: ==, ===, !=, !==, >, <, >=, <=
5 == "5";    // true (loose equality - type coercion)
5 === "5";   // false (strict equality - no type coercion)
5 != "5";    // false
5 !== "5";   // true

// Logical operators: && (and), || (or), ! (not)
true && false;  // false
true || false;  // true
!true;          // false

// Nullish coalescing operator: ?? (returns right side if left is null/undefined)
null ?? 'default';  // 'default' (if null/undefined)
```

1.5 Control Statements

If-Else:

```
if (age >= 18) {
    console.log("Adult");
} else if (age >= 13) {
    console.log("Teenager");
} else {
    console.log("Child");
}

// Ternary
var status = age >= 18 ? "Adult" : "Minor";
```

Switch:

```
switch (day) {
    case 0:
        dayName = "Sunday";
        break;
    case 1:
        dayName = "Monday";
        break;
    default:
        dayName = "Unknown";
}
```

Loops:

```
// For
for (var i = 0; i < 5; i++) {
    console.log(i);
}

// While
while (i < 5) {
    console.log(i);
    i++;
}

// Do-While
do {
    console.log(i);
    i++;
} while (i < 5);

// For...in (objects)
for (var key in obj) {
    console.log(key + ": " + obj[key]);
}

// For...of (iterables)
for (var item of arr) {
    console.log(item);
}

// forEach
arr.forEach(function(item, index) {
    console.log(index + ": " + item);
});
```

1.6 Arrays

```
var fruits = ["Apple", "Banana", "Orange"];

// Access - get elements by index or length
fruits[0];           // "Apple"
fruits.length;       // 3

// Mutators - modify the array
fruits.push("Mango");           // Add to end
fruits.pop();                   // Remove from end
fruits.unshift("Grape");        // Add to start
fruits.shift();                 // Remove from start
fruits.splice(1, 1, "Kiwi");   // Replace elements

// Iterators - loop through array and transform data
fruits.forEach((item, index) => {});
fruits.map(item => item.toUpperCase());
fruits.filter(item => item.length > 5);
fruits.reduce((acc, item) => acc + item, "");

// Search - find elements in array
fruits.indexOf("Banana");      // 1
fruits.includes("Mango");      // true
fruits.find(item => item.startsWith("A"));
```

1.7 Functions

```
// Function declaration - named function that can be called before definition
function greet(name) {
    return "Hello, " + name;
}

// Function expression - function assigned to a variable
var greet = function(name) {
    return "Hello, " + name;
};

// Arrow function - concise syntax, 'this' is lexically bound
var greet = (name) => "Hello, " + name;
var greet = (name) => { return "Hello, " + name; };

// Default parameters - default value when argument is not provided
function greet(name = "Guest") {
    return "Hello, " + name;
}

// Rest parameters - collect multiple arguments into an array
function sum(...numbers) {
    return numbers.reduce((a, b) => a + b, 0);
}

// Callback - function passed as argument to another function
function callback(x) { return x * 2; }
[1, 2, 3].map(callback);
```

1.8 JavaScript Objects

```
// Object creation
var person = {
    name: "John",
    age: 25,
    email: "john@example.com",
    greet: function() {
        return "Hello, I'm " + this.name;
    }
};

// Accessing properties - using dot notation or bracket notation
person.name;           // John
person["age"];        // 25
person.greet();        // "Hello, I'm John"

// Adding/modifying properties
person.city = "New York";
person.age = 26;

// Constructor - function to create objects
function Person(name, age) {
    this.name = name;
    this.age = age;
}
var p1 = new Person("John", 25);

// ES6 Class - modern syntax for creating objects
class Person {
    constructor(name, age) {
        this.name = name;
        this.age = age;
    }

    greet() {
        return "Hello, I'm " + this.name;
    }
}
```

1.9 Built-in Objects

String:

```
var str = "Hello World";
str.length;                      // 11
str.toUpperCase();               // "HELLO WORLD"
str.toLowerCase();               // "hello world"
str.indexOf("World");            // 6
str.substring(0, 5);            // "Hello"
str.slice(-5);                  // "World"
str.split(" ");                 // ["Hello", "World"]
str.replace("World", "PHP");    // "Hello PHP"
str.trim();                      // Remove whitespace
```

Math:

```
Math.round(4.5);    // 5
Math.floor(4.9);    // 4
Math.ceil(4.1);     // 5
Math.abs(-5);       // 5
Math.max(1, 5, 3);  // 5
Math.min(1, 5, 3);  // 1
Math.random();       // Random 0-1
Math.pow(2, 3);     // 8
Math.sqrt(16);      // 4
```

Date:

```
var date = new Date();
date.getFullYear();     // 2024
date.getMonth();        // 0-11
date.getDate();         // 1-31
date.getHours();        // 0-23
date.getMinutes();      // 0-59
date.getTime();         // Timestamp
```

1.10 DOM (Document Object Model)

```
// Select elements - find DOM elements
document.getElementById("myId");
document.getElementsByClassName("myClass");
document.getElementsByTagName("p");
document.querySelector(".myClass");
document.querySelectorAll("p");

// Modify content - change element's HTML or text
element.innerHTML = "<p>New content</p>";
element.textContent = "New text";
element.setAttribute("class", "highlight");
element.getAttribute("id");

// Modify styles - change CSS properties
element.style.color = "blue";
element.style.display = "none";
element.classList.add("active");
element.classList.remove("active");
element.classList.toggle("active");

// Create elements - add new nodes to DOM
var div = document.createElement("div");
div.textContent = "Hello";
document.body.appendChild(div);

// Event handling - respond to user actions
element.addEventListener("click", function(event) {
    console.log(event.target);
});
element.addEventListener("mouseover", handler);
```

1.11 Form Validation

```
function validateForm() {
    var name = document.forms["myForm"]["name"].value;
    var email = document.forms["myForm"]["email"].value;
    var password = document.forms["myForm"]["password"].value;

    if (name == "") {
        alert("Name must be filled out");
        return false;
    }

    if (!validateEmail(email)) {
        alert("Valid email required");
        return false;
    }

    if (password.length < 8) {
        alert("Password must be at least 8 characters");
        return false;
    }

    return true;
}

function validateEmail(email) {
    var re = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    return re.test(email);
}
```

1.12 call(), apply(), bind()

```
var person = {
    name: "John",
    greet: function(city) {
        return "Hello, " + this.name + " from " + city;
    }
};

var anotherPerson = { name: "Jane" };

// call()
person.greet.call(anotherPerson, "New York");  // "Hello, Jane from New York"

// apply()
person.greet.apply(anotherPerson, ["New York"]);  // "Hello, Jane from New York"

// bind()
var greetJane = person.greet.bind(anotherPerson, "New York");
greetJane();  // "Hello, Jane from New York"
```

---

2. JSP (JavaServer Pages)

2.1 Definition and Overview

What is JSP?

JavaServer Pages (JSP) is a server-side technology that simplifies the creation of dynamic web pages in Java. JSP allows developers to embed Java code directly into HTML pages, making it easy to create dynamic content that can interact with databases, process user input, and perform server-side operations.

Why JSP is Important:

1. **Easy Integration with Java:**
    - Leverages Java's powerful features
    - Access to Java libraries
    - Object-oriented programming
2. **Separation of Concerns:**
    - Designers can work on HTML
    - Developers work on Java code
    - Clean separation of presentation and logic
3. **Built-in Features:**
    - Implicit objects (request, response, session)
    - Custom tag libraries
    - Expression language
4. **Enterprise Ready:**
    - Used in large-scale applications
    - Integration with Java EE
    - Security features

How JSP Works:

1. User requests a JSP page
2. Web container checks if JSP needs translation
3. JSP is translated to a servlet (Java class)

4. Servlet is compiled to bytecode
5. Container executes the servlet
6. HTML response is generated and sent to browser

2.2 Anatomy of a JSP Page

Every JSP page has specific components that work together:

1. Directives: Provide instructions to the JSP container about how to process the page.

```
<%@ page language="java" contentType="text/html" %>
<%@ include file="header.jsp" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
```

2. Declarations: Declare variables and methods that can be used throughout the page.

```
<%!
    private int visitCount = 0;
    public String getCurrentTime() {
        return new java.util.Date().toString();
    }
%>
```

3. Scriptlets: Contain Java code that executes during page processing.

```
<%
    String name = request.getParameter("name");
    if (name != null) {
        out.println("Welcome, " + name);
    }
%>
```

4. Expressions: Evaluate and output Java expressions directly to the page.

```
<p>Current time: <%= new java.util.Date() %></p>
<p>2 + 2 = <%= 2 + 2 %></p>
```

5. Actions: Standard XML-style tags that perform specific operations.

```
<jsp:forward page="next.jsp" />
<jsp:include page="header.jsp" />
```

2.3 JSP Processing - The Lifecycle

Phase 1: Translation:

- JSP page is parsed
- Converted to a servlet Java source file
- Container generates the servlet class

Phase 2: Compilation:

- Java servlet source is compiled to bytecode
- Class file is created

Phase 3: Initialization:

- jspInit() method is called
- One-time setup operations

Phase 4: Request Processing:

- _jspService() method handles each request
- Generated HTML sent to client

Phase 5: Destruction:

- jspDestroy() called when page is removed
- Cleanup operations

2.4 JSP Directives - In Detail

Page Directive: Controls page properties and settings.

```
<%@ page
    language="java"
    contentType="text/html; charset=UTF-8"
    import="java.util.*, java.sql.*"
    errorPage="error.jsp"
    isErrorPage="false"
    buffer="8kb"
    autoFlush="true"
    session="true"
    isThreadSafe="true"
%>
```

Include Directive: Includes content at translation time (static include).

```
<%@ include file="header.jsp" %>
```

- File included before translation
- Good for static content (headers, footers)
- Shares variables with including page

Taglib Directive: Declares custom tag library for use in the page.

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<%@ taglib prefix="fmt" uri="http://java.sun.com/jsp/jstl/fmt" %>
```

2.5 JSP Implicit Objects - Complete Reference

JSP provides eight implicit objects that are automatically available:

Object	Class	Purpose
request	HttpServletRequest	Client request data
response	HttpServletResponse	Server response
out	JspWriter	Output to response
session	HttpSession	User session
application	ServletContext	Application scope
config	ServletConfig	Servlet configuration
pageContext	PageContext	Page-scoped context
page	Object	Reference to servlet

Detailed Usage:

1. **request Object:**

```
<%
String name = request.getParameter("name");
String method = request.getMethod();
String ip = request.getRemoteAddr();
String header = request.getHeader("User-Agent");
%>
```

2. **response Object:**

```
<%
response.setContentType("text/html");
response.setCharacterEncoding("UTF-8");
response.sendRedirect("login.jsp");
response.setStatus(HttpServletResponse.SC_OK);
%>
```

3. **session Object:**

```
<%
session.setAttribute("user", userObj);
User user = (User)session.getAttribute("user");
String sessionId = session.getId();
%>
```

4. **application Object:**

```
<%
application.setAttribute("siteName", "My Website");
String configParam = application.getInitParameter("adminEmail");
%>
```

2.6 JSP Scope Objects - Understanding Visibility

Page Scope:

- Data exists only within the current JSP page
- Uses pageContext implicit object
- Default scope for most operations
- Data lost when page finishes

Request Scope:

- Data available during single HTTP request
- Used when forwarding/including between pages
- Accessible via request object
- Data persists across include/forward

Session Scope:

- Data persists across multiple requests
- Unique per user session
- Typical for login status, preferences
- Destroyed when session expires

Application Scope:

- Data available to all users/pages

- Shared across entire application
- Exists for application lifetime
- Use sparingly (memory impact)

2.7 Using Beans in JSP - Enterprise Pattern

JavaBeans are reusable Java components that follow specific conventions:

- Public no-argument constructor
- Properties accessed via get/set methods
- Serializable (optional)

Bean Scope in JSP:

Scope	Description
page	Current page only
request	Current request
session	User session
application	Whole application

Working with Bean Properties:

```
<!-- Set property automatically from request parameter -->
<jsp:setProperty name="user" property="*" />

<!-- Set specific property -->
<jsp:setProperty name="user" property="email" value="new@email.com" />

<!-- Get property -->
<jsp:getProperty name="user" property="name" />
```

2.8 Session Tracking in JSP - Complete Implementation

Why Session Tracking Matters:

HTTP is stateless, but web applications need to track users:

- Login/logout functionality

- Shopping carts
- User preferences
- Security

Method 1: Hidden Form Fields:

```
<!-- Form with hidden session ID -->
<form action="process.jsp" method="post">
    <input type="hidden" name="sessionId" value="<%= session.getId() %>">
    <!-- other fields -->
</form>
```

Method 2: URL Rewriting:

```
<%
String encodedURL = response.encodeURL("checkout.jsp");
%>
<a href="<%= encodedURL %>">Checkout</a>
```

Method 3: Cookies (Most Common):

```
<%
// Automatic session tracking (enabled by default)
String sessionId = session.getId();

// Create custom cookie - store data in user's browser
Cookie userCookie = new Cookie("username", "john");
userCookie.setMaxAge(3600); // 1 hour (in seconds)
userCookie.setPath("/");
response.addCookie(userCookie);
%>
```

Method 4: HTTPSession API (Recommended):

```
<%
// Session is automatically created
HttpSession hs = request.getSession(true);

hs.setAttribute("cart", cart);
hs.setMaxInactiveInterval(1800); // 30 minutes

// Check if new session
if (hs.isNew()) {
    // New visitor
}

// Invalidate session (logout)
hs.invalidate();
%>
```

2.9 Connecting to Database in JSP - JDBC

Database Architecture in JSP:

```
Browser → JSP → JDBC Driver → Database
         ← Response ← ResultSet ←
```

Complete Example with Error Handling:

```
<%@ page import="java.sql.*" %>
<%
Connection conn = null;
PreparedStatement pstmt = null;
ResultSet rs = null;

try {
    // Load driver
    Class.forName("com.mysql.cj.jdbc.Driver");

    // Get connection
    String url = "jdbc:mysql://localhost:3306/mydb";
    conn = DriverManager.getConnection(url, "root", "password");

    // Query with PreparedStatement (prevents SQL injection)
    String sql = "SELECT * FROM users WHERE email = ?";
    pstmt = conn.prepareStatement(sql);
    pstmt.setString(1, "user@example.com");

    rs = pstmt.executeQuery();

    // Process results
    while (rs.next()) {
        out.println(rs.getString("name") + "<br>");
    }

} catch (ClassNotFoundException e) {
    out.println("Driver not found");
} catch (SQLException e) {
    out.println("Database error: " + e.getMessage());
} finally {
    // Always close resources
    if (rs != null) rs.close();
    if (pstmt != null) pstmt.close();
    if (conn != null) conn.close();
}
%>
```

Why Use PreparedStatement?

- Prevents SQL injection attacks
- Improves performance for repeated queries
- Handles special characters automatically

2.2 Anatomy of a JSP Page

```
<%@ page language="java" contentType="text/html; charset=UTF-8" %>
<%@ page import="java.util.*" %>
<%@ page errorPage="error.jsp" %>
<%@ include file="header.jsp" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>

<!DOCTYPE html>
<html>
<head>
    <title>JSP Page</title>
</head>
<body>
    <h1>Welcome to JSP</h1>

    <%-- JSP Comment --%>

    <%!
        // Declaration (instance variables/methods)
        int count = 0;
        String getMessage() {
            return "Hello from JSP!";
        }
    %>

    <%
        // Scriptlet (Java code)
        String name = "John";
        int a = 10, b = 20;
        int sum = a + b;
    %>

    <%-- Expression (output) --%>
    <p>Name: <%= name %></p>
    <p>Sum: <%= sum %></p>
    <p>Message: <%= getMessage() %></p>

    <%-- JSP Action --%>
    <jsp:forward page="next.jsp" />
</body>
</html>
```

2.3 JSP Directives

Directive	Description
`page`	Page attributes (import, contentType, errorPage, isELIgnored)
`include`	Include file at translation time
`taglib`	Define custom tag library

```
<%@ page import="java.util.*, java.io.*" buffer="8kb" autoFlush="true" %>
<%@ include file="header.jsp" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
```

2.4 JSP Implicit Objects

Object	Type	Description
`request`	HttpServletRequest	Client request (request scope)
`response`	HttpServletResponse	Server response
`out`	JspWriter	Output stream
`session`	HttpSession	User session (session scope)
`application`	ServletContext	Application scope
`config`	ServletConfig	Servlet configuration
`pageContext`	PageContext	Page context (page scope)
`page`	Object	This servlet instance

2.5 JSP Scripting Elements

Element	Tag	Purpose
Declaration	`<%! %>`	Define variables and methods
Scriptlet	`<% %>`	Execute Java code
Expression	`<%= %>`	Output Java expressions

2.6 Connecting to Database in JSP

```
<%@page import="java.sql.*"%>
<%
String url = "jdbc:mysql://localhost:3306/TestDb";
String username = "root";
String password = "root";

Connection con = null;
Statement stmt = null;
ResultSet rs = null;

try {
    Class.forName("com.mysql.jdbc.Driver");
    con = DriverManager.getConnection(url, username, password);
    stmt = con.createStatement();
    String query = "SELECT * FROM students";
    rs = stmt.executeQuery(query);

    while(rs.next()) {
        out.println(rs.getInt("id") + " - " + rs.getString("name"));
    }
} catch(Exception e) {
    out.println("Error: " + e.getMessage());
} finally {
    if(rs != null) rs.close();
    if(stmt != null) stmt.close();
    if(con != null) con.close();
}
%>
```

2.7 Using Beans in JSP

```
<jsp:useBean id="user" class="com.example.UserBean" scope="session"/>
<jsp:setProperty name="user" property="name" value="John"/>
<jsp:setProperty name="user" property="email" value="john@example.com"/>
<jsp:getProperty name="user" property="name"/>
```

UserBean.java:

```java
package com.example;
public class UserBean {
    private String name;
    private String email;

    public UserBean() {}

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
}
```

2.8 Session Tracking in JSP

Using Sessions:

```jsp
<%
session.setAttribute("username", "John");
session.setAttribute("userRole", "admin");
String user = (String)session.getAttribute("username");
%>
```

Using Cookies:

```jsp
<%
Cookie cookie = new Cookie("user", "John");
cookie.setMaxAge(3600);  // 1 hour
cookie.setPath("/");
response.addCookie(cookie);

Cookie[] cookies = request.getCookies();
if (cookies != null) {
    for (Cookie c : cookies) {
        if (c.getName().equals("user")) {
            out.println(c.getValue());
        }
    }
}
%>
```

2.9 JSP Scope Objects

Scope	Description	Accessibility
`page`	Current JSP page only	Page only
`request`	Current request	Forward/include between pages
`session`	User session	Across requests
`application`	Whole application	All users

Unit II: Practice Questions

1. **[Easy]** Explain the difference between `var`, `let`, and `const` in JavaScript.
2. **[Medium]** Write a JavaScript function to validate an email address using regular expressions and test password strength (min 8 chars, at least 1 uppercase, 1 lowercase, 1 number).
3. **[Hard]** Create a JSP page that connects to a MySQL database and displays user information in an HTML table with pagination (10 records per page).

Unit II: Quick Reference Card

Concept	Syntax/Formula	Example
Variable	`let x = 5`	Block-scoped
Function	`function() {}`	Arrow: `() => {}`
Array	`[]`	`[1, 2, 3]`
Object	`{}`	`{key: value}`
DOM Query	`document.getElementById()`	Select element
JSP Scriptlet	`<% code %>`	Java in HTML
JSP Expression	`<%= expr %>`	Output to page
JSP Declaration	`<%! code %>`	Variables/methods
JSP Bean	`<jsp:useBean>`	Use Java bean
Session	`session.setAttribute()`	Store user data
Cookie	`new Cookie()`	Client storage
Scope	page/request/session/application	Data visibility

Unit I & II Summary

Key Differences: HTML vs CSS vs JavaScript vs JSP

Feature	HTML	CSS	JavaScript	JSP
Type	Markup	Style	Script	Server Script
Execution	Browser	Browser	Browser	Server
Database	No	No	No	Yes (JDBC)
Security	Client visible	Client visible	Client visible	Server-side

Key Concepts to Remember

1. **HTML:** Structure of web page
2. **CSS:** Presentation and styling
3. **JavaScript:** Client-side interactivity
4. **JSP:** Server-side dynamic content
5. **XML:** Data interchange format
6. **DTD/Schema:** XML validation

End of Document